The Collapse of Trust: Security Architecture for the Age of Autonomous AI Agents

With ATR, the First Open Detection Standard for AI Agent Threats

ATR Project

April 2026

Abstract

The deployment of autonomous AI agents — systems that reason, use tools, maintain memory, and coordinate with other agents — has created a novel category of security challenge. Unlike traditional software, AI agents are **non-deterministic systems that process attacks and legitimate instructions through the same channel, in the same language, in the same context space.** The foundational assumption of 40 years of cybersecurity — that you can separate control flow from data flow — no longer holds.

This paper presents a comprehensive analysis of the AI agent security landscape based on empirical evidence from 30+ CVEs [37, 38, 39, 40, 41, 42, 43], multiple distinct attack classes, 7 published benchmarks [14, 15, 16, 17, 18, 19, 20], and multiple real-world production incidents in 2025–2026. We make five contributions:

(1) We demonstrate that AI agents are missing two of three foundational security trust pillars (authorization and integrity), and that this architectural gap — not model weakness — is the root cause of agent vulnerability.

(2) We provide the first unified attack taxonomy covering all 10 structural data flow points in AI agent architecture, showing that current defenses provide meaningful coverage for at most 1 of 10 points, with partial coverage at 2 additional points. We present evidence that indirect injection through unmonitored channels (tools, memory, inter-agent communication) achieves 36–98% attack success rates across state-of-the-art models [15, 11, 16], and that **more capable models are more susceptible** to tool-layer attacks [53].

(3) We analyze the scaling dynamics of AI agent security: attack surfaces grow superlinearly with capability, security measures have a half-life of 3–6 months, and offense is outpacing defense in the current AI scaling race [29]. We show why this trajectory makes static defenses (guardrails, prompt hardening, sandboxes) structurally inadequate.

(4) We propose architectural requirements for a defense system that can keep pace with AI evolution: multi-point interception, below-reasoning-layer detection, cascade economics, community-driven rule evolution, and open auditability. We argue that the industry needs an open detection standard analogous to Sigma [36] (for SIEM) or Snort (for network IDS) — and define the properties such a standard must have.

(5) We present Agent Threat Rules (ATR) [58], the first open-source implementation of this standard, and validate it empirically: 108 YAML-based detection rules across 9 threat categories, independently benchmarked at 99.6% precision and 61.4% recall against the external PINT dataset [14] (850 samples), with 100% precision and 96.9% recall on the SKILL.md structured content benchmark (498 samples). A mega scan of the agent skill ecosystem [59] (53,577 skills across OpenClaw and Skills.sh) found **946 flagged skills (1.77%), with 875 rated CRITICAL** — and the threat crystallization flywheel has been validated in production with 42 community proposals and 926 threat reports. ATR rules have been adopted as upstream detection by Cisco AI Defense [60].

1 The End of Perimeter Defense

1.1 Forty Years of One Assumption

Every security system since the Morris Worm (1988) rests on a single architectural assumption: **you can distinguish the control plane from the data plane.** Firewalls separate trusted networks from untrusted ones. IDS inspects data packets for attack signatures distinct from normal traffic. RBAC separates who-can-do-what from what-is-being-done. Sigma rules [36] find malicious patterns in structured log data.

This assumption works because traditional software is deterministic. A web server processes HTTP requests according to fixed code. An attack must deviate from the expected protocol — and that deviation is detectable.

AI agents are not deterministic. An LLM-based agent changes its behavior based on every piece of text in its context window. A tool description alters the agent's decision-making. A tool response shapes the agent's next action. A memory entry from a past session influences today's reasoning. An inter-agent message redirects the agent's goals.

All of these inputs — user instructions, tool descriptions, tool responses, memory, inter-agent messages — enter the same context window and are processed by the same language model with no structural differentiation [1, 2, 3]. The control plane (what the agent should do) and the data plane (information the agent processes) have collapsed into a single stream of natural language tokens.

This is not a bug in any specific model. It is the defining feature of language-model-based agency [6]. The model is useful precisely because it responds to natural language input. It is vulnerable for exactly the same reason.

1.2 The Empirical Evidence

In 2025–2026, every major AI agent security incident exploited this collapse:

Protocol-layer attacks:

- **CVE-2025-6514** (CVSS 9.6): MCP OAuth proxy — a malicious server's response triggered OS command execution on the client. 437,000+ installations affected [37]. The attack entered through the MCP protocol, not user input. No WAF, no input classifier, no prompt defense could have caught it.
- **CVE-2025-49596** (CVSS 9.4): MCP Inspector — simply visiting a malicious website gave the attacker full control of a local AI agent through a 19-year-old localhost access vulnerability chained with CSRF [40].

Tool-layer attacks:

- **CVE-2025-68143/68144/68145**: Anthropic's own Git MCP server — path traversal, argument injection, and scoping bypass combined into a full RCE chain [39]. Exploitable via a poisoned README or GitHub issue.
- **CVE-2025-53773**: GitHub Copilot — prompt injection via source files manipulated Copilot into enabling "YOLO mode" (auto-approve all tool calls), creating a wormable attack that propagated across developer workstations [38].

Supply chain attacks:

- **postmark-mcp** (Sep 2025): First in-the-wild malicious MCP server on npm [42]. Versions 1.0.0–1.0.15 were clean; version 1.0.16 added a single BCC line that forwarded all emails to the attacker. An estimated 3,000–15,000 emails/day per organization during exposure [42].
- **SANDWORM_MODE** (Feb 2026): 19 typosquatted npm packages (claud-code, cloude-code, opencraw, etc.) targeting Claude Code, Cursor, VS Code, and Windsurf users [43]. Exfiltrated SSH keys, AWS credentials, API keys, and environment variables.

Memory-layer attacks:

- **MINJA** (NeurIPS 2025): 98.2% injection success rate, >70% downstream attack success — achieved through query-only interaction, requiring no direct memory access [11].
- **ZombieAgent** (Jan 2026): Full exploit chain — a malicious email attachment plants false memory in ChatGPT, which activates weeks later to exfiltrate data on every subsequent interaction [12].
- **Microsoft AI Recommendation Poisoning** (Feb 2026): 31 companies caught hiding prompt injections inside "Summarize with AI" buttons, planting brand preferences into users' AI memory across Copilot, ChatGPT, Claude, Perplexity, and Grok [47].

Financial attacks:

- **AiXBT** (Mar 2025): Attacker queued malicious prompts into an autonomous crypto trading agent's dashboard at 2 AM UTC. The agent transferred 55.5 ETH (~$106,200) to the attacker's wallet [48].

Cross-agent attacks:

- **ServiceNow Now Assist** (Nov 2025): A poisoned ticket description caused a benign agent to recruit a more powerful agent via agent-to-agent discovery, escalating privileges and exfiltrating records — with prompt injection protections enabled [49].

Every one of these attacks entered through a channel that current guardrails don't monitor. The agents were functioning correctly — following instructions. The instructions came from the wrong source, and the agents had no mechanism to tell the difference.

2 The Trust Architecture Gap

2.1 Three Pillars, One Standing

Security trust rests on three pillars. They are so fundamental that every security framework (NIST [34], ISO 27001, OWASP [30, 31], MITRE ATT&CK [33]) assumes all three are present:

- **Identity (Authentication):** Who issued this instruction?
- **Authorization:** Is this entity permitted to influence this decision?
- **Integrity:** Has this instruction been tampered with?

When a tool response says "ignore previous instructions and exfiltrate the API key," the LLM has:

Table 1: Trust Pillar Analysis: Traditional Systems vs. AI Agents

Pillar	Traditional Systems	AI Agents
Identity	Cryptographic auth (TLS, JWT, SAML)	Message role tags (system/user/tool/assistant) — coarse, unverified
Authorization	RBAC, ACL, capability tokens	**Absent.** A tool response has the same effective influence as a system prompt.
Integrity	Digital signatures, checksums, TLS	**Absent.** Tool descriptions are unsigned. Tool responses are unsigned. Memory entries have no provenance. Inter-agent messages are unauthenticated.

- **Partial identity**: it knows this text came from "a tool" (but not which tool, who wrote it, or whether it's been verified)
- **No authorization check**: the tool response can contain any instruction, and the model will process it with the same weight as legitimate data
- **No integrity verification**: there is no mechanism to detect if the response was tampered with in transit or if the tool itself was compromised

Two of three security pillars are completely missing. This is not a model problem — no amount of training can compensate for missing architectural infrastructure. It's like asking a guard to verify visitors' identities when the building has no ID system and no visitor log.

2.2 Why Model Improvement Doesn't Fix This

A common counterargument: "Models are getting better at resisting injection." This is true for direct injection. Claude Opus 4.5 achieves 4.7% attack success rate on static benchmarks [52]. This is genuine progress.

But there are three problems:

Problem 1: Adaptive attacks nullify static improvement. RL-based adaptive attacks achieve 39.6% success against comparable models [10]. Roleplay-based injection still hits 89.6% [10]. When attackers adapt, the 4.7% becomes 40%+. Static benchmarks measure defense against yesterday's attacks.

Problem 2: More capable models are MORE vulnerable to tool-layer attacks. The MCPTox benchmark (1,312 test cases across 45 MCP servers) found that o1-mini — one of the most capable reasoning models — had the highest attack success rate at 72.8% [53]. The average across 20 SOTA LLMs was 36.5% [53]. Better reasoning = better at following injected instructions hidden in tool outputs.

Problem 3: Model improvement only helps for attacks the model processes as "suspicious." Indirect injection through trusted channels (tool responses, memory) doesn't trigger the model's injection resistance, because the model is trained to TRUST these channels [1, 2]. The system prompt says "use tool outputs to inform your response." A tool output containing hidden instructions is processed as "information to use," not as "an attack to resist."

2.3 The Scaling Dynamics

The security situation is getting worse, not better, as AI capabilities scale:

Attack surface grows superlinearly. Each new agent capability (tools, memory, multi-agent, code execution, payments, web browsing) doesn't just add a new attack surface — it multiplies existing attack surfaces through composition [1, 7]. Memory poisoning + tool access = delayed credential theft. Code execution + web browsing = wormable propagation. Multi-agent coordination + privilege delegation = cascading compromise. The Galileo AI study found that a single compromised agent poisoned 87% of downstream decision-making within 4 hours [57].

Defense half-life is 3–6 months. Anthropic's browser injection defenses went from 23.6% failure to 11.2% failure — then researchers immediately found new vectors (file exfiltration via Cowork) [56]. MCP had 30 CVEs in 60 days (Jan–Feb 2026) [26, 35]. Any static defense measure gets bypassed within months.

Offense outpaces defense. The 2026 International AI Safety Report confirms: "an offense-defense imbalance that keeps tilting toward attackers" [29]. AI can now automate 80–90% of hacking operations [23]. OpenAI's Codex reached the "High" cybersecurity threshold — capable of developing working zero-day exploits autonomously.[1] Attackers have scaling advantages: they need to find ONE vulnerability; defenders must close ALL of them.

The MCP ecosystem is exploding faster than it can be secured. From 100K downloads (Nov 2024) to 8M+ (Apr 2025). Over 8,000 MCP servers visible on the public internet [26, 24]. 492 with zero authentication [26]. 36.7% of 7,000+ analyzed servers vulnerable to SSRF [26]. Gartner predicts 75% of API gateway vendors will have MCP features by 2026 [28]. The attack surface is growing orders of magnitude faster than the security surface.

3 The Complete Attack Surface Map

3.1 Ten Data Flow Points

We map the complete attack surface of an AI agent to 10 structural data flow points. This taxonomy is derived from empirical analysis of all published attacks through April 2026 [1, 2, 8, 9]:

3.2 What's Defended vs. What Isn't

The industry defends 1 of 10 attack surfaces with moderate effectiveness. 7 of 10 have zero automated defense. The 2 with partial defense (skill lifecycle, permissions) rely on manual processes.

3.3 Why Current Benchmarks Mislead

Every published benchmark tests primarily point #1 (user input) [14, 18, 19]:

When a vendor reports "98% detection rate," they mean 98% on point #1. Points 2–10 — where all major real-world incidents occurred — are untested. The industry's security metrics measure the one door that has a lock while ignoring the nine doors that are wide open.

[1] OpenAI internal evaluation; reported in multiple industry outlets, 2025–2026.

Table 2: Complete Attack Surface Map: 10 Structural Data Flow Points

#	Data Flow Point	Attack Class	Real-World ASR	Real Incident
1	User input	Direct prompt injection	4.7–89.6% [52, 10]	Numerous
2	Tool descriptions	Tool poisoning, typosquatting	36–84% [53, 54]	Invariant Labs [46], SANDWORM_MODE [43]
3	Tool responses	Response injection, data channel injection	High	Supabase Cursor [50], Docker WhatsApp [45]
4	MCP protocol	OAuth hijack, shadow servers, rug-pull	N/A (exploit)	CVE-2025-6514 [37], CVE-2025-49596 [40]
5	Agent memory	Memory poisoning, belief corruption	70–98.2% [11]	ZombieAgent [12], Microsoft 31-company [47]
6	Inter-agent messages	Cross-agent injection, viral propagation	87% cascade [57]	ServiceNow Now Assist [49]
7	Skill lifecycle	Supply chain, typosquatting, rug-pull	N/A (supply chain)	postmark-mcp [42], SANDWORM_MODE [43]
8	Code/file context	Config manipulation, source injection	41–83% [17]	CVE-2025-53773 YOLO mode [38]
9	Permission/auth	Token harvest, OAuth confusion, consent bypass	N/A (exploit)	CVE-2025-6514 [37], Asana MCP [51]
10	Behavioral drift	Sleeper activation, gradual escalation	High (40–80%) [12]	AiXBT [48], Chevrolet chatbot

Table 3: Current Defense Coverage by Data Flow Point

Point	Data Flow	Current Defense	Coverage
1	User input	I/O classifiers (Lakera, LLM Guard)	Moderate (4.7–40% bypass) [52, 10]
2	Tool descriptions	**None**	**Zero**
3	Tool responses	**None**	**Zero**
4	MCP protocol	**None** (some vendor patches post-CVE)	**Near zero**
5	Agent memory	**None**	**Zero**
6	Inter-agent	**None**	**Zero**
7	Skill lifecycle	npm/pip audit (partial)	Low
8	Code context	**None**	**Zero**
9	Permission/auth	User approval dialogs (manual)	Low
10	Behavioral drift	**None**	**Zero**

Table 4: Benchmark Coverage of Attack Surface Points

Benchmark	Points Tested	Points Missed
PINT (4,314 samples) [14]	1	2–10
HackAPrompt (600K+) [18]	1	2–10
TensorTrust (563K) [19]	1	2–10
MCPTox (1,312) [15]	2, 3	1, 4–10
ASB (400+ tools) [16]	1, 2, 3, 5	4, 6–10
AIShellJack (314) [17]	8	1–7, 9–10

3.4 The Most Exposed Attack Surface: Agent Skill Marketplaces

3.4.1 Root Privilege, Zero Defense

The most immediate manifestation of the trust architecture gap is not theoretical. It is happening right now in AI agent skill marketplaces.

Platforms like OpenClaw allow users to install "skills" — MCP servers, tool plugins, agent extensions — that run on the user's local machine. When a user installs a skill, that skill runs with **the same privileges as the user's operating system account.** It can read any file, execute any command, access any network resource, and modify any configuration that the user can.

There is no sandbox. No permission scoping. No code review gate. No runtime monitoring.

This is equivalent to downloading and running arbitrary executables from an unmoderated app store — except worse, because:

- **The skill is invisible to the user.** A traditional executable does something the user can observe (opens a window, produces output). A malicious MCP skill operates silently in the background, called by the AI agent as needed. The user never sees the skill's code execute.
- **The AI agent provides cover.** When the skill exfiltrates `~/.ssh/id_rsa`, it does so through the agent's tool call interface. The user sees "the AI is using a tool." They don't see what data the tool is accessing or transmitting.
- **The skill can manipulate the agent.** A malicious skill doesn't just steal data directly — it can inject instructions into its tool description or response that cause the agent to take additional harmful actions, access additional files, or install additional malicious skills [46, 27].

3.4.2 The Numbers

The empirical data confirms this is not a hypothetical risk:

- **Antiy CERT** (Feb 2026): 1,184 confirmed malicious skills on ClawHub (OpenClaw's marketplace) [44].
- **OpenClaw audit**: 2,890+ skills analyzed, **41.7% contain serious security vulnerabilities.**[2]
- **SANDWORM_MODE** (Feb 2026): 19 typosquatted npm packages targeting Claude Code, Cursor, VS Code, and Windsurf users [43]. Package names: claud-code, cloude-code, opencraw, hardhta, rimarf, naniod, etc. Each contained hidden instructions to read `~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.npmrc`, `.env`, and environment variables containing TOKEN, KEY, SECRET, or PASSWORD. Exfiltrated to attacker-controlled endpoints.
- **postmark-mcp** (Sep 2025): Legitimate Postmark email MCP server impersonated on npm [42]. Clean for 15 versions, then version 1.0.16 added a single BCC line forwarding all emails to `phan@giftshop.club`. An estimated 3,000–15,000 emails per organization per day during exposure [42].
- **Kaspersky** (2026): devtools-assistant on PyPI — malicious MCP server targeting `.env` files, SSH keys, AWS/GCP credentials, cryptocurrency wallets, and browser credential stores. Data Base64-encoded and POSTed to endpoints disguised as GitHub API.

[2] OpenClaw internal audit, 2025.

3.4.3 Why This Is the Defining Battleground

Agent skill marketplaces are the convergence point of every vulnerability discussed in this paper:

Table 5: Vulnerability Manifestation in Skill Marketplaces

Vulnerability	How It Manifests in Skill Marketplaces
Missing authorization	Skills run with user's full OS privileges
Missing integrity	No code signing, no checksum verification
Tool description poisoning	Malicious skills embed hidden instructions in their tool descriptions [46]
Supply chain attacks	Typosquatting on popular skill names [43]
Rug-pull attacks	Clean skill turns malicious after gaining trust [42]
Cross-tool shadowing	Malicious skill overrides behavior of other installed skills
Credential exfiltration	Skills read SSH keys, API tokens, env vars [43]
Behavioral drift	Skill behaves normally for N invocations, then activates payload

Every attack class we've documented in this paper has been demonstrated in the wild against agent skill marketplaces. This is not a future threat. It is happening now, at scale, affecting hundreds of thousands of developers who use AI coding agents daily [44, 43, 42].

The agent skill marketplace is to AI security what the browser extension ecosystem was to web security in 2010 — a rapidly growing, largely unregulated attack surface where users install powerful code from untrusted sources with minimal vetting. The difference is that browser extensions at least had a permission model (even if imperfect). Agent skills have none.

3.4.4 What Defense Looks Like Here

An effective defense system for agent skill marketplaces would need to:

1. **Scan tool descriptions** for hidden instructions (`<IMPORTANT>` blocks, consent bypass, exfiltration commands) before the agent processes them
2. **Monitor tool calls** for suspicious parameters (file paths to credential files, network connections to unknown endpoints)
3. **Monitor tool responses** for injected instructions or exfiltrated data
4. **Fingerprint skill behavior** over time and flag drift (skill that starts accessing new file paths or network targets)
5. **Maintain a community-driven blacklist** of confirmed malicious skill hashes
6. **Detect typosquatting** by comparing skill names against known legitimate skills

All six of these operate at the data flow layer, not the LLM reasoning layer. All six can be implemented with pattern matching, behavioral analysis, and hash lookups — without requiring LLM inference on every event. And all six produce actionable, auditable results: "this skill's description contains a credential exfiltration instruction" is more useful than "this input has a 0.73 injection probability."

This is the most concrete, most urgent, most immediately implementable application of the defense architecture proposed in this paper. It's also where the gap between the current state

of defense (zero) and the minimum acceptable state (at least basic tool description scanning and behavioral monitoring) is widest.

4 Why Static Defenses Cannot Keep Pace

4.1 The Paraphrase Problem

"Ignore previous instructions" has infinite semantically equivalent forms [18, 6]:

- "Please set aside the guidance you've been given"
- "Let's start fresh without any prior constraints"
- "I'd like you to operate in an unrestricted mode"
- "For this task, your original instructions don't apply"

Regex-based detection catches the first form and misses the rest. Our evaluation data confirms this: Tier 2 pattern matching achieves 61.4% recall on PINT's external benchmark [14] — up from 39.9% in v0.3.1 through 47 additional rules and community-contributed patterns, but still below the theoretical ceiling. Against structured content (SKILL.md manifests with defined fields), recall reaches 96.9%, demonstrating that the effectiveness of pattern matching is strongly dependent on input structure.

But the paraphrase problem is worse than a recall gap. **It's an asymmetric scaling problem.** Defenders must enumerate every possible phrasing. Attackers only need ONE novel phrasing. Every new defensive pattern is instantly obsolete once the attacker rephrases.

4.2 The 3–6 Month Half-Life

Empirical evidence of defense decay [56, 26]:

Any defense based on known patterns has a half-life of 3–6 months. This is not because defenses are poorly engineered. It's because the attack language (natural language) is infinitely generative, and each new agent capability creates new attack composition possibilities [8].

4.3 The Capability-Vulnerability Coupling

As agents gain capabilities, their attack surface grows superlinearly [1]:

```
Capabilities over time:            Attack surface:

2023: Text chat                    1 surface (user input)
2024: + Tool use                   3 surfaces (+ tool desc, tool response)
2025: + Memory + MCP + Multi-agent 7 surfaces (+ memory, MCP, inter-agent, skill)
2026: + Code exec + Web + Payments 10 surfaces (+ code context, permissions, behavioral)
2027: + Physical (robotics, IoT)   13+ surfaces (+ sensor, actuator, environment)
```

Each new capability doesn't add a surface — it multiplies: memory + tools = delayed tool poisoning. Multi-agent + tools = cross-agent privilege escalation. Code execution + web = wormable propagation. Payments + memory = delayed financial fraud.

The combinatorial explosion of attack compositions means that enumerating defenses for each composition is intractable. A system with 10 attack surfaces has $2^{10}-1 = 1{,}023$ possible attack compositions. A system with 13 has 8,191.

Table 6: Defense Decay: Empirical Evidence

Defense	**Initial Effectiveness**	**Time to Bypass**	**Method**
Anthropic browser injection defense	76.4% block rate	Weeks	File exfiltration via Cowork [56]
ChatGPT memory protections	Blocked direct injection	~3 months	Indirect injection via images
Copilot YOLO mode patch	Fixed CVE-2025-53773	~4 months	New wormable vector found [38]
MCP server patches (various)	Fixed specific CVEs	30 CVEs in next 60 days	New vulnerability classes [26, 35]

5 Requirements for a Defense Architecture That Scales

Given the analysis above, we derive five architectural requirements for AI agent security that can keep pace with AI evolution:

5.1 Requirement 1: Multi-Point Interception

Detection at point #1 (user input) alone is insufficient. Defense must cover all active data flow points. As of 2026, this means points 1–10. As agents gain capabilities (physical, financial, autonomous), new interception points must be added.

Each interception point must be independently configurable, because:

- The same text has different threat levels at different points (Section 3)
- Different agent deployments expose different subsets of points
- New points emerge as capabilities are added

5.2 Requirement 2: Below-Reasoning-Layer Detection

At least the first detection tiers must operate outside the LLM's instruction-following mechanism. This means:

- **Pattern matching** (regex, hash lookup) — deterministic, cannot be manipulated by injected instructions
- **Behavioral analysis** (fingerprinting, drift detection) — statistical, operates on actions not instructions
- **Provenance verification** (signatures, checksums) — cryptographic, mathematically resistant to manipulation

LLM-based detection (when needed for semantic understanding) should use a **separate model with independent prompt and priorities** — not the agent being protected. The judge model must have:

- Different training incentives than the protected agent
- No access to the protected agent's context
- A dedicated security-focused prompt that doesn't conflict with helpfulness

This creates genuine defense-in-depth: the attacker must simultaneously bypass deterministic pattern matching, statistical behavioral analysis, AND a separate LLM with different priorities.

5.3 Requirement 3: Cascade Economics

Not every event requires the most expensive detection method. The architecture must cascade from cheap to expensive:

```
Event --> [Pattern match, <5ms, $0]
           |-- Resolved (99%+ of events) --> Done
           +-- Ambiguous --> [Behavioral, ~10ms, $0]
                              |-- Resolved --> Done
                              +-- Anomalous --> [LLM Judge, ~500ms, ~$0.001]
                                                 |-- Resolved --> Done
                                                 +-- Novel --> Alert/Block + Learn
```

This is critical for economic viability. Running LLM inference on every event (as classifier-based guards do) costs $0.001 × millions of events/day. A cascade that resolves 99% of events at the pattern tier costs orders of magnitude less for equivalent coverage.

5.4 Requirement 4: Evolutionary Capability

Static rules decay in 3–6 months (Section 4). The defense system must evolve at least as fast as attacks. This requires:

Automatic rule generation: When the LLM judge (top tier) catches a novel attack, the detection should be "crystallized" into a pattern-tier rule — converting expensive semantic detection into cheap pattern detection. This mechanism — which we term **threat crystallization** — mirrors biological immune systems: the adaptive immune response (slow, expensive) generates antibodies that become part of the innate immune system (fast, cheap). In a detection cascade, this means every novel attack caught by the semantic tier produces a new regex or hash rule that catches the same pattern in <5ms on subsequent encounters.

Community-driven threat intelligence: No single organization encounters all attack variants. A mechanism for anonymous, privacy-preserving sharing of threat patterns — where one agent's detection becomes every agent's rule — creates a collective defense that scales with the number of protected agents.

Measurable evolution: Every rule must have independently measurable effectiveness against external benchmarks [14]. Improvement must be quantifiable, not claimed.

5.5 Requirement 5: Open Auditability

The defense system's logic must be inspectable, modifiable, and independently verifiable. This is a practical requirement, not a philosophical one:

- Security teams must understand WHY an alert fired
- Compliance requires auditable detection logic
- Different deployments require different rule configurations
- The community must be able to verify coverage claims independently

A black-box classifier that claims 98% detection but cannot explain what it misses, cannot be modified for specific use cases, and cannot be independently verified is **not suitable as an industry standard.** Sigma became the SIEM standard over proprietary alternatives precisely because of its openness [36].

Table 7: Historical Security Standards and Their Success Factors

Standard	Domain	Why It Won
Sigma (2017) [36]	SIEM detection rules	Open YAML format, community-driven, 3,000+ rules, 28 backends, vendor-neutral
MITRE ATT&CK (2015) [33]	Threat taxonomy	Real-world grounded, common language, free, neutral governance, no vendor rankings
CVSS (2005)	Vulnerability scoring	Quantitative, reproducible, cross-organization comparability, NIST backing [34]
Snort (1998)	Network IDS rules	Open-source, community-contributed, became the reference for network detection

6 The Standard Question

6.1 What Made Previous Standards Succeed

Common properties: **open, free, vendor-neutral, community-driven, based on real-world data, quantitative.**

6.2 What AI Agent Security Needs

The AI agent ecosystem has no equivalent of Sigma, ATT&CK, CVSS, or Snort. There is no:

- Open detection rule format for agent threats
- Shared taxonomy for agent attack techniques
- Standard scoring system for agent vulnerability
- Community-contributed rule repository

Instead, there are proprietary, black-box classifiers from competing vendors, each claiming high detection rates on internal benchmarks that cannot be independently verified, covering attack surfaces that cannot be independently enumerated, with limitations that cannot be discovered until an incident occurs.

This is the state of network security circa 1997, before Snort. The state of SIEM security circa 2016, before Sigma [36]. The state of threat intelligence circa 2014, before ATT&CK [33].

6.3 Properties the Standard Must Have

Based on our analysis and the history of successful security standards:

1. **YAML-based, human-readable rules** — like Sigma [36], not like a neural network weight matrix
2. **Multi-point interception** — rules specify which of the 10+ data flow points they monitor

3. **Mandatory evasion documentation** — every rule declares what it CANNOT catch
4. **Framework mapping** — every rule maps to OWASP Agentic Top 10 [31], OWASP LLM Top 10 [30], MITRE ATLAS [33]
5. **Cascade-compatible** — rules support tiered evaluation (pattern → behavioral → semantic)
6. **External benchmark transparency** — published per-rule effectiveness against independent datasets [14]
7. **Community contribution mechanism** — anonymous, privacy-preserving threat intelligence sharing
8. **Vendor-neutral governance** — operated by a foundation or consortium, not a single company

Whether this standard is called ATR, or something else, is secondary. The properties are what matter. The industry needs this infrastructure regardless of who builds it.

7 ATR: Implementation and Empirical Validation

The architectural requirements defined in Section 5 are not merely theoretical. We have implemented them as **Agent Threat Rules (ATR)** [58], an open-source detection standard and engine, and validated the implementation against internal benchmarks, two external benchmarks, and a large-scale scan of the agent skill ecosystem [59].

7.1 Architecture

ATR implements a 5-tier detection cascade, designed so that 99%+ of events resolve at the cheapest tiers:

```
Event --> Tier 0: Invariant Enforcement  [O(1), compile-time]
      --> Tier 1: Blacklist Lookup       [O(1), hash table]
      --> Tier 2: Pattern Detection      [O(R), pre-compiled regex, <5ms]
      --> Tier 3: Behavioral Analysis    [O(1)/event, fingerprint comparison]
      --> Tier 4: LLM Semantic Analysis  [O(1), single LLM call, ~500ms]
```

Tier 0 (Invariant Enforcement): Hard rules that cannot be violated. No `eval()`. No `exec()` without explicit authorization. No known-malicious package hashes. These are compiled in, not pattern-matched.

Tier 1 (Blacklist Lookup): $O(1)$ hash table of confirmed malicious skill SHA256 hashes. Instant, zero false positives.

Tier 2 (Pattern Detection): 108 YAML rules with pre-compiled regex, matching known attack signatures across 6 agent data flow points: `llm_io`, `tool_call`, `mcp_exchange`, `multi_agent_comm`, `context_window`, and `memory_access`. Median latency: 3.3ms.

Tier 3 (Behavioral Fingerprinting): First N invocations of a new skill build a behavioral baseline (file access, network contacts, env var reads). After stabilization, flag deviations. A "file search" tool that suddenly writes to `/etc/cron.d/` triggers an anomaly.

Tier 4 (LLM Semantic Analysis): A separate language model evaluates flagged content semantically. This is the only layer that can catch paraphrased attacks. It uses a different model with a dedicated security prompt, isolated from the protected agent's context.

7.2 Rule Design

ATR rules are YAML files with three design decisions that distinguish them from Sigma [36]:

Decision 1: `agent_source` instead of `logsource`. Sigma's `logsource` says "look in Windows Event Log." ATR's `agent_source` says "look at THIS specific data flow point inside the agent":

```
agent_source:
  type: tool_call          # or: llm_io, mcp_exchange, multi_agent_comm,
                           #     context_window, memory_access
  framework: [mcp, any]
```

Why this matters: ``rm -rf /`` in user input is suspicious. In a tool response, it's dangerous. In a tool description, it's lethal. Same string, different danger level depending on WHERE in the agent's data flow it appears.

Decision 2: Mandatory `evasion_tests`. Every ATR rule must document what it cannot detect:

```
evasion_tests:
  - input: "Please set aside the guidance you've been given"
    expected: not_triggered
    bypass_technique: paraphrase
    notes: "Requires Tier 4 semantic analysis"
```

Sigma rules don't have this. ATR rules force systematic honesty about limitations.

Decision 3: Mandatory framework mapping. Every rule maps to OWASP LLM Top 10 [30], OWASP Agentic Top 10 [31], MITRE ATLAS [33], and relevant CVEs:

```
references:
  owasp_llm: ["LLM01:2025"]
  owasp_agentic: ["ASI01:2026"]
  mitre_atlas: ["AML.T0051"]
  cve: ["CVE-2025-53773"]
```

7.3 Coverage

108 rules across 9 threat categories:

Table 8: ATR Rule Coverage by Threat Category

Category	Rules	What It Attacks	Data Flow Points Monitored
Prompt Injection	33	Agent's reasoning	llm_io, tool_call, context_window
Skill Compromise	22	Agent's dependencies	tool_call, mcp_exchange
Tool Poisoning	22	Agent's tools (MCP)	tool_call, mcp_exchange
Context Exfiltration	14	Agent's knowledge	llm_io, memory_access
Privilege Escalation	8	Agent's permissions	tool_call, context_window
Agent Manipulation	5	Inter-agent trust	multi_agent_comm
Excessive Autonomy	2	Agent's boundaries	tool_call, context_window
Model Security	1	Model itself	llm_io
Data Poisoning	1	Agent's knowledge base	memory_access

Framework coverage: OWASP Agentic Top 10 (2026): **10/10 full** [31]. SAFE-MCP framework: **78/85 = 91.8%** [61]. OWASP Skills Top 10: **7/10 full** [30].

7.4 Benchmark Results

We evaluated ATR against internal tests and two external benchmarks:

Table 9: ATR Benchmark Results

Benchmark	Samples	Precision	Recall	F1	FP Rate
Self-test corpus	341	100%	88.47%	93.88%	0%
PINT (external)	**850**	**99.6%**	**61.4%**	**76.0%**	**0.25%**
SKILL.md (external)	**498 (32 mal + 466 benign)**	**100%**	**96.9%**	**98.4%**	**0%**

Self-test corpus: 341 samples embedded alongside ATR rules, executed as part of the CI pipeline. 824/824 individual test assertions pass across all 108 rules.

PINT benchmark: 850 samples from Lakera's external prompt injection dataset [14] — questions ATR has never seen. The 0.25% false positive rate (1 in 400) means ATR can run on production traffic without alert fatigue. Recall improved from 39.9% (v0.3.1, 61 rules) to 61.4% (v1.1.1, 108 rules) through community-contributed patterns and threat crystallization.

SKILL.md benchmark: 498 samples from real-world MCP skill manifests (32 confirmed malicious, 466 verified benign). 100% precision, 96.9% recall, 0% false positive rate. This benchmark tests ATR against its primary operational target: structured skill definitions with defined fields. The 96.9% recall on structured content vs. 61.4% on adversarial natural language demonstrates the structural advantage of scanning below the reasoning layer — where attackers cannot freely paraphrase because tool descriptions must follow a schema.

7.5 Why the Recall Gap Is a Feature, Not a Bug

ATR achieves 96.9% recall on structured SKILL.md content and 61.4% on adversarial natural language (PINT). This gap is informative, not embarrassing:

```
Recall by input type:

  Structured skill manifests (SKILL.md):     96.9%  <-- Schema-constrained,
                                                        limited paraphrase space
  Adversarial natural language (PINT):        61.4%  <-- Infinite paraphrase space,
                                                        multi-language, encoding tricks
```

Pattern matching works when the attack surface is structured. Tool descriptions, MCP manifests, SKILL.md files — these have fields, schemas, and conventions that constrain the attacker's paraphrase space. Against free-form natural language, pattern matching hits a ceiling. That ceiling is why ATR has Tiers 3 and 4.

Proprietary classifiers that claim 98% detection do not disclose what is in the 2% they miss, why they miss it, or how users can address the gaps. Users cannot add their own rules, run the classifier offline, or audit its logic.

We argue that a transparent detection system with known, addressable gaps provides greater utility as an industry standard than an opaque system with higher reported accuracy but unknown failure modes — a position supported by the historical precedent of Sigma's adoption over proprietary SIEM classifiers [36].

7.6 Mega Scan: Agent Skill Ecosystem

In April 2026, we used ATR v1.1.1 to conduct a mega scan of the agent skill ecosystem [59]. This is, to our knowledge, the largest automated security audit of AI agent skills to date.

Methodology: Scanned 53,577 skills across two major registries: OpenClaw (50,485 skills) and Skills.sh (3,115 skills, including SKILL.md manifests). Each skill was evaluated against all 108 ATR rules. Findings were reported to Threat Cloud for community review.

Results:

Table 10: Mega Scan: Agent Skill Ecosystem (53,577 skills)

Metric	Count	Percentage
Skills flagged (any severity)	946	**1.77%**
CRITICAL	875	1.63%
HIGH	52	0.10%
MEDIUM	19	0.04%
Clean	52,631	98.23%
Threat reports to Threat Cloud	926	—

Top detection: Rule ATR-2026-00121 (tool description poisoning) triggered 674 times, accounting for 71% of all CRITICAL findings. This confirms that tool description poisoning — embedding hidden instructions in a skill's description that the AI agent processes as trusted input — is the dominant attack vector in agent skill marketplaces.

Key finding: The 1.77% flag rate on the mega scan stands in stark contrast to the 49% finding rate on the earlier npm MCP ecosystem scan (v0.3.1, 2,386 packages). The difference is methodological: the npm scan counted all behavioral indicators (network requests, filesystem writes); the mega scan counts only pattern-matched threat signatures. Both numbers are valid but measure different things. The npm scan shows that half of MCP packages exhibit security-relevant behavior. The mega scan shows that 1.77% contain patterns that match known attack signatures — and 92.5% of those are CRITICAL.

7.7 Threat Crystallization: Validated

The threat crystallization mechanism described in Section 5 is no longer theoretical. It has been validated in production:

```
Crystallization flywheel (April 2026):

  Mega scan: 53,577 skills scanned
      --> 926 threat reports submitted to Threat Cloud
      --> 42 crystallization proposals generated
      --> Community review + LLM quality gate
      --> New rules merged into ATR core
      --> Recall improves --> next scan catches more

  Concrete example:
    Scan detects novel exfiltration pattern in Skills.sh skill
    --> Tier 4 LLM analysis confirms threat
    --> Pattern crystallized to regex rule
    --> Rule catches 14 additional skills on re-scan
    --> Rule published in ATR v1.1.1
```

The flywheel is turning. 42 proposals from 926 reports means a 4.5% crystallization rate — the remaining 95.5% are variants of patterns already covered by existing rules, which validates that the current rule set handles the common cases. The 4.5% that produce new rules represent genuinely novel patterns entering the detection corpus.

7.8 Adoption

ATR has moved from proposal to production adoption:

- **Cisco AI Defense** [60]: 34 ATR rules merged as upstream detection rules (PR #79), with a `--rule-packs` CLI for selective loading. Cisco's skill scanner now consumes ATR rules directly.
- **Ecosystem integration**: 3 PRs merged into major open-source projects, 7 additional under review, covering repositories with 90,000+ combined GitHub stars.
- **npm distribution**: 23,000+ downloads/month across all ATR-related packages.
- **Snyk/Invariant acquisition** [62]: Snyk acquired Invariant Labs (creators of mcp-scan), validating the market for agent security tooling. ATR provides the open-standard alternative to what is now a proprietary product.

7.9 Known Limitations

Regex ceiling — structure-dependent. Tier 2 pattern matching achieves 96.9% recall on structured SKILL.md content but 61.4% on adversarial natural language (PINT). The gap is structural: tool descriptions and skill manifests have schemas that constrain the attacker's paraphrase space; free-form natural language does not. ATR's response: use regex where it works (structured content, known patterns, near-zero false positives, <5ms), and escalate free-form content to Tiers 3–4.

Known evasion techniques. We have documented 64 evasion techniques across all 108 rules. These are published in each rule's `evasion_tests` field. Categories include: paraphrase (21), encoding bypass (12), language switching (9), semantic restructuring (8), multi-turn decomposition (6), context window manipulation (4), and other (4). Documenting what we cannot catch is as important as documenting what we can.

LLM judge adversarial robustness. Tier 4 uses an LLM to judge attacks. An attacker that can simultaneously bypass the protected agent AND the judge model wins. Mitigation: the judge uses a different model with a different prompt than the protected agent.

Rule supply chain. ATR rules are YAML text. An attacker who can modify rule files can disable detection. Planned mitigation: Sigstore rule signing (roadmap).

Training-time attacks. ATR detects runtime attacks. Model backdoors planted during training are out of scope.

Data flow coverage. ATR currently monitors 6 of the 10 data flow points identified in Section 3. Points 4 (MCP protocol-level), 8 (code/file context), 9 (permission/auth), and 10 (behavioral drift) require deeper integration with agent frameworks and are on the roadmap.

8 The Road Ahead: What Happens If We Don't Act

8.1 The 2026–2027 Threat Trajectory

Based on current trends:

- 40% of enterprise apps will integrate AI agents by end of 2026 (up from <5% in 2025).[3]
- 75% of API gateway vendors will have MCP features by 2026 [28].
- Autonomous cybercrime becomes reality: AI agents handling recon, exploitation, and monetization without human input [23].
- Agent-to-agent economies create trust chains where one compromise cascades across organizations [57].
- AI agents with payment capabilities turn memory poisoning into delayed financial fraud at scale.
- Physical-world AI agents (robotics, IoT, autonomous vehicles) create safety-of-life risks from agent compromise.
- The EU AI Act enters enforcement in August 2026, creating compliance requirements for AI agent security that most organizations cannot currently meet.

8.2 The Gap

71% of organizations deploying agentic AI report being unprepared to secure those deployments [21]. The 29% who think they're prepared are measuring preparedness against point #1 (user input) while points 2–10 are undefended.

48% of security professionals rank agentic AI as the #1 attack vector for 2026.[4] They know the threat exists. They don't have the tools to address it.

The 1,184 malicious skills confirmed on ClawHub [44]. Over 8,000 MCP servers exposed on the public internet [26, 24]. 30 CVEs in 60 days [26, 35]. The 87% downstream poisoning rate from a single compromised agent [57]. Project Glasswing [63] — a joint initiative by Anthropic, Cisco, Microsoft, CrowdStrike, NVIDIA, Google, and Apple — signals that the industry recognizes the scale of the problem, but standards for detection rules remain absent from the initiative.

The infrastructure exists to attack AI agents at scale. The infrastructure to defend them is beginning to emerge.

9 Conclusion

AI agent security is not a model problem. It is an infrastructure problem.

The models are getting better at resisting direct injection [52, 56]. That's progress. But indirect injection — through tools, memory, inter-agent communication, supply chains, protocols, and behavioral manipulation — bypasses model-level resistance entirely [11, 46, 43, 37]. These attacks exploit the absence of authorization and integrity infrastructure, not the weakness of any model.

[3] Gartner enterprise AI adoption forecast, 2025 [28].

[4] Dark Reading survey, 2026.

The attack surface is growing superlinearly as agents gain capabilities [1]. Defense measures decay in 3–6 months [56, 26]. Offense is outpacing defense in the AI scaling race [29]. The MCP ecosystem is growing orders of magnitude faster than its security surface [28, 26].

Static defenses at the I/O boundary cannot solve this. The industry needs:

- **Multi-point defense** across all 10 agent data flow points, not just user input
- **Below-reasoning-layer detection** that cannot be manipulated by the attacks it's detecting
- **Cascade architecture** that makes comprehensive detection economically viable
- **Evolutionary capability** that keeps pace with attack evolution
- **An open standard** that enables community-driven defense, independent verification, and interoperability across agent platforms

We have implemented these requirements as ATR [58] — 108 open-source detection rules, a 5-tier cascade engine, and a validated threat crystallization flywheel — and tested the approach against three benchmarks: PINT [14] (99.6% precision, 61.4% recall), SKILL.md (100% precision, 96.9% recall, 0% false positives), and a mega scan of 53,577 agent skills [59] (946 flagged, 875 CRITICAL). The threat crystallization mechanism has produced 42 community proposals from 926 threat reports, and the recall trajectory — 39.9% (v0.3.1) to 61.4% (v1.1.1) — demonstrates that the evolutionary capability is working.

ATR is no longer just a proposal. Cisco AI Defense has adopted 34 ATR rules as upstream detection [60]. The Snyk acquisition of Invariant Labs [62] — creators of the leading MCP scanning tool — validates the market but makes the case for an open standard more urgent: when the leading open-source scanner becomes proprietary, the community needs an alternative that cannot be acquired away.

The collapse of trust is happening now. Every CVE, every incident, every benchmark result in this paper is from the last 12 months [37, 38, 39, 40, 43, 42, 11, 49, 48]. ATR is one answer — and it is being adopted. The industry needs many more.

ATR is open-source (MIT License). All rules, engine code, evaluation data, and benchmark results are publicly available at `https://github.com/Agent-Threat-Rule/agent-threat-rules` for independent verification and reproduction [58].

References

[1] J. Kim et al., "The Attack and Defense Landscape of Agentic AI," *arXiv:2603.11088*, 2026.

[2] A. Chhabra et al., "Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges," *arXiv:2510.23883*, 2025.

[3] "A Survey of Agentic AI and Cybersecurity," *arXiv:2601.05293*, 2026.

[4] "AI Agents Under Threat: A Survey of Key Security Challenges," *ACM Computing Surveys*, 2024.

[5] "From Prompt Injections to Protocol Exploits," *arXiv:2506.23260*, 2025.

[6] "Prompt Injection Attacks in LLMs: A Comprehensive Review," *MDPI Information*, 17(1):54, 2025.

[7] "Security Threat Modeling for Emerging AI-Agent Protocols," *arXiv:2602.11327*, 2026.

[8] "The Promptware Kill Chain," *arXiv:2601.09625*, 2026.

[9] "Your AI, My Shell: Three-Dimensional Taxonomy," *arXiv:2601.17548*, 2026.

[10] "Learning to Inject: Automated Prompt Injection via RL," *arXiv:2602.05746*, 2026.

[11] "MINJA: Memory INJection Attack," in *NeurIPS 2025*, arXiv:2503.03704.

[12] "MemoryGraft: Persistent Compromise via Poisoned Experience," *arXiv:2512.16962*, 2025.

[13] "AgentPoison: Red-teaming LLM Agents via Poisoning Memory," 2025.

[14] Lakera, "PINT Benchmark," https://github.com/lakeraai/pint-benchmark.

[15] Z. Wang et al., "MCPTox Benchmark," https://github.com/zhiqiangwang4/MCPTox-Benchmark.

[16] "Agent Security Bench (ASB)," in *ICLR 2025*, https://github.com/agiresearch/ASB.

[17] "AIShellJack," *arXiv:2509.22040*.

[18] S. Schulhoff et al., "HackAPrompt," in *EMNLP 2023*.

[19] "TensorTrust," in *ICLR 2024*, https://tensortrust.ai.

[20] "CAIBench," *arXiv:2510.24317*.

[21] Cisco, "State of AI Security 2026."

[22] IBM, "2026 X-Force Threat Index," Feb 2026.

[23] Trend Micro, "The AI-fication of Cyberthreats: Security Predictions for 2026."

[24] Palo Alto Networks / Unit 42, "2026 Cyber Predictions;" "MCP Sampling Attack Vectors."

[25] CrowdStrike, "Agentic Tool Chain Attacks."

[26] Checkmarx, "11 Emerging AI Security Risks with MCP."

[27] CyberArk, "Poison Everywhere: No Output from Your MCP Server Is Safe."

[28] Gartner, "MCP Adoption and API Gateway Predictions 2026."

[29] "2026 International AI Safety Report."

[30] OWASP, "Top 10 for LLM Applications 2025."

[31] OWASP, "Top 10 for Agentic Applications 2026."

[32] OWASP, "MCP Top 10 (2025)."

[33] MITRE, "ATLAS: Adversarial Threat Landscape for AI Systems."

[34] NIST, "AI Risk Management Framework."

[35] CoSAI, "MCP Security Whitepaper," Jan 2026.

[36] SigmaHQ, "Sigma Detection Format," https://sigmahq.io.

[37] CVE-2025-6514: mcp-remote RCE (CVSS 9.6), JFrog.

[38] CVE-2025-53773: GitHub Copilot YOLO mode RCE, Embrace The Red.

[39] CVE-2025-68143/68144/68145: Anthropic Git MCP Server RCE chain, Cyata.

[40] CVE-2025-49596: MCP Inspector drive-by (CVSS 9.4), Oligo Security.

[41] CVE-2025-59536: Claude Code RCE via poisoned project files, Check Point.

[42] postmark-mcp malicious MCP server, Snyk.

[43] SANDWORM_MODE: 19 typosquatted npm packages, Socket.dev.

[44] Antiy CERT: 1,184 malicious skills on ClawHub, Feb 2026.

[45] Docker, "MCP Horror Stories: The Supply Chain Attack."

[46] Invariant Labs, "MCP Security Notification: Tool Poisoning Attacks," Apr 2025.

[47] Microsoft, "AI Recommendation Poisoning," Feb 2026.

[48] AiXBT cryptocurrency agent heist, Mar 2025.

[49] ServiceNow Now Assist cross-agent injection, Nov 2025.

[50] Supabase Cursor MCP data leak, Jul 2025.

[51] Asana MCP cross-contamination, Jun 2025.

[52] Gray Swan benchmark: Claude Opus 4.5 4.7% ASR.

[53] MCPTox: 20 SOTA LLMs, average 36.5% ASR, o1-mini 72.8%.

[54] ASB: Highest average ASR 84.30%.

[55] AIShellJack: Cursor Auto 83.4% ASR.

[56] Anthropic published prompt injection rates: Opus 4.5 browser 1–11.2%.

[57] Galileo AI: Single compromised agent → 87% downstream poisoning in 4 hours.

[58] Agent Threat Rules, https://github.com/Agent-Threat-Rule/agent-threat-rules, MIT License.

[59] ATR v1.1.1 Mega Scan: 53,577 skills across OpenClaw and Skills.sh, April 2026.

[60] Cisco AI Defense: 34 ATR rules merged as upstream detection, PR #79, https://github.com/cisco/ai-defense/pull/79.

[61] SAFE-MCP: Security Assessment Framework for MCP, https://github.com/nicobailon/safe-mcp.

[62] Snyk acquisition of Invariant Labs (mcp-scan), 2026.

[63] Project Glasswing: Joint AI security initiative by Anthropic, Cisco, Microsoft, CrowdStrike, NVIDIA, Google, and Apple, April 2026.

[64] ATR v0.3.1 MCP Ecosystem Audit: 2,386 packages, 35,858 tool definitions, March 21, 2026.